Session 1 — Foundations 1

Linux essentials (the command line)

Qiuyu Yu, Human Development and Family Science (HDFS), Auburn University

Audience: beginners, no prior command-line experience assumed.
Goal: by the end you can navigate the filesystem, edit and run a script, set permissions, and read a program's output.

Roadmap for today

1. **Why the command line** (mindset)
2. **Terminal, shell, and environment**
3. **Navigation & paths**
4. **Files, folders & scripting building blocks**
5. **Variables, arguments & permissions**
6. **Redirection, pipes, and getting help**
7. **Exit codes & chaining**

1. Why the command line (mindset)

- Linux is an operating system with **no clicking**; everything is a typed command.
- Trade-off: the first couple of weeks are painful, then it is far faster and fully scriptable.
- Most neuroimaging tools (AFNI, fMRIPrep, FSL) are **script-based**: transparent, reproducible, no manual GUI steps.
- A GUI walks you through each step by hand: easy to start, but slow and error-prone to repeat. The command line scripts the same work, so it runs fast and identically across many files, though one wrong line can hit every file at once.

1b. Terminal, shell, and environment

- **Terminal** = the window you type into. **Shell** (usually **bash**) = the program inside it that reads and runs your commands.
- Your **environment** = settings that live in *this* session: variables, your `PATH`, and any modules you loaded.
- **`PATH`** = the list of folders the shell searches for a command. "command not found" means the command is not on your `PATH`.
- Every new terminal starts a **fresh shell**, hence a fresh environment. That is why you reload modules and re-activate conda each time you log in.
- To run something automatically on every new shell, put it in `~/.bashrc`.

2a. Navigation & paths

```
pwd                 # print the current directory
cd /work/lab        # go to an absolute path
cd ..               # up one level
cd ~                # home directory
cd -                # back to the previous directory
ls -l               # long listing: permissions, size, date
ls -aR dir/         # all files, recursively into subfolders
```

- `~` = home, `.` = here, `..` = one level up

2b. Absolute vs relative paths

- **Absolute** path starts at `/` and is the same everywhere: `/home/you/data/sub01`
- **Relative** path is read from where you currently are: `../data/sub01`, `./run.sh`
- The #1 beginner error is a wrong path. "No such file or directory" almost always means *you are not where you think you are*, so check `pwd`.

2c. Files & folders

```
mkdir -p a/b/c          # make nested folders in one go
cp file dest/           # copy a file
cp -r dir/ dest/        # copy a folder and its contents
mv old new              # move OR rename
touch script.sh         # create an empty file
find . -name "*.nii*"   # find files by pattern
rm file                 # delete a file
rm -rf dir/             # delete a folder (DANGER: no undo, no trash)
```

- **Tip:** read these as **from -> to** (source first, destination second): `cp file dest/`, `mv old new`.
- The wildcard `*` matches anything: `cp sub-*_T1w.nii.gz dest/`

2d. Scripting building blocks

```
sub=sub01                  # set a variable (no spaces around =)
echo $sub                  # use it with a leading $

for i in $(seq -w 1 12); do
  mkdir sub$i              # sub01, sub02, ... sub12
done

if [[ -e sub01 ]]; then
  echo "already exists"
else
  mkdir sub01
fi
```

- A script is just these commands saved in a `.sh` file.

2e. Variables, arguments & placeholders

```
sub=sub01              # set a variable (no spaces around =)
echo $sub              # use it with a leading $
export SUBJ=sub01      # export = make it visible to programs/scripts you launch
```

- A script reads **arguments by position**: inside `run.sh`, `$1` is the first word you typed.
  - `./run.sh sub01` -> `$1` is `sub01`

2f. Permissions

```
ls -l                    # shows  rwx r-x r-x  owner group others
ls -l my_script.sh       # check permissions of a specific file
ls -ld my_data_dir/      # check the folder itself, not its contents

chmod +x script.sh       # make a script executable
chmod -R g+rwx dir/      # give your group access, recursively
```

- **Permissions & ownership (files vs. directories):**
  - `ls -l` shows the permissions, owner, and group of files.
  - `ls -ld` checks a directory itself, rather than its contents.
- **r** (read = 4), **w** (write = 2), **x** (execute = 1), for **owner / group / others**.
  - The three digits of `chmod 755` are the sum of those values, one digit per category.

2f2. chmod in practice

Read a numeric mode as three digits: **owner / group / others**.

- `chmod 755 script.sh` -> full control for you (`7`), read/execute for group and others (`5`). Standard for shared scripts.
- `chmod 770 -R data/` -> full access for you and your lab group, nothing for other cluster users (`0`). Good for shared project data.
- `chmod 777 -R dir/` -> read/write/execute for *everyone* on the cluster.
  - **Warning:** avoid this on shared HPC; it is a security risk.
- "Permission denied" when running `./script.sh` means you forgot `chmod +x` (or `chmod 755`).

2g. Redirection & pipes (read this before troubleshooting)

```
mycmd >  out.txt          # send normal output to a file (overwrite)
mycmd >> out.txt          # append instead of overwrite
mycmd 2> err.txt          # send errors (stderr) to a file
mycmd &> all.txt          # send BOTH output and errors
mycmd | grep -i error     # pipe output into another command
```

- Reading a long log: `less file`, `tail -f file`, `grep -i error file`
- This is how you capture and search a job's output later.
- Same syntax works **inside a `.sh` script** too: redirection and `|` are shell features, not a terminal-only trick.

2g2. Redirection with SLURM: who does it, and where?

Same syntax as above. The only question is **where the redirection happens**. Three cases:

1. **sbatch does it for you:** once submitted, the whole script's stdout + stderr go to `slurm-%j.out`. You write nothing.
2. **You write it inside the job script:** to pull one step out on its own, `python step1.py 2> step1_err.log` keeps that step's errors out of the main log.
3. **You type it live at the terminal:** `srun` / login-node testing has no default log, so capture it by hand with `my_test_cmd > test_output.txt`; watch a running job with `tail -f slurm-%j.out`.

2g3. Other uses of redirection

- **Build a text file:** make a subject list with `ls sub-* > subjects.txt`, then feed it to a loop or an array job.
- **Write a clean summary** to its own file, kept separate from the noisy main log.
- **Silence chatty tools:** `noisy_cmd > /dev/null` throws the output away so it does not bloat the log.

(SLURM itself comes in Session 2; here just note where its output goes.)

2h. Survival keys & getting help

- `Tab` auto-completes file and command names (type less, fewer typos)
- `Up` arrow / `Ctrl+R` recall and search past commands
- `Ctrl+C` cancel a running command; `Ctrl+L` or `clear` clears the screen
- Read what a command does: `man ls` or `ls --help`
- When lost: `pwd` (where am I), `ls` (what's here), `less file` (look inside)

2i. Exit codes & chaining

Every command returns an **exit code**: `0` means success, non-zero means failure.

```
some_command ; echo $?      # $? holds the last command's exit code
cmd1 && cmd2                # run cmd2 only if cmd1 succeeded
cmd1 || echo "failed"       # run the right side only if cmd1 failed
```

- This is how scripts stop on the first failure instead of charging ahead with bad data.

Assignment (Session 1)

1. Log into a terminal (your laptop's shell, WSL, or the HPC — Session 2 covers logging in)
2. Practice moving around: `pwd`, `cd`, `ls -l`, and tell absolute from relative paths
3. Make a folder tree with `mkdir -p`, copy/move/rename a few files, then clean up
4. Write a tiny `.sh` script with a `for` loop, `chmod +x` it, and run it with an argument (`./run.sh sub01`)
5. Redirect a command's output to a file and search it with `grep`

Reading: https://qiuyuyu3.github.io/fMRI-Data-Processing-Manual-on-HPC/introduction/

You can now...

- explain why neuroimaging lives on the command line
- navigate paths, and tell absolute from relative
- create/copy/move files, write a small script, and set permissions
- capture a command's output and read its exit code

Next: get onto the HPC, submit real jobs, and set up your developer tools.